

Diseño y Análisis de Algoritmos - INFO145

Guía de Ejercicios - Unidad I.

Académico: Héctor Ferrada

1. Problemas de Rendimiento Asintótico

1. Un particular algoritmo de ordenamiento ofrece un tiempo de $T(n) = O(n \log n)$ tardando $1 \mu s$ en ordenar 10^3 elementos. Como el tiempo de ordenamiento es directamente proporcional a $n \log n$ ($\log$ en base 10), es decir $T(n) = c \cdot n \log n$ para alguna constante $c > 0$, encuentre una fórmula explícita para $T(n)$ y estime cuanto tardaría este método en ordenar $N = 10^6$ items.
2. Los algoritmos A y B requieren exactamente $T_A(n) = 0,1n^2 \log_{10} n$ y $T_B(n) = 2,5n^2 \mu s$, respectivamente, para resolver un problema de tamaño n . En base a esto:
 - Seleccione el algoritmo que es asintóticamente mejor y determine n_0 tal que para cualquier $n > n_0$ el algoritmo seleccionado tenga también un mejor rendimiento que el otro algoritmo (Asuma que todo es ejecutado en la misma máquina).
 - Si el tamaño de la entrada a este problema es $n \leq 10^9$ ¿Cuál algoritmo usted recomienda utilizar?
3. Los algoritmos A y B requieren exactamente $T_A(n) = c_1 n \log_2 n$ y $T_B(n) = c_2 n^2 \mu s$, respectivamente, para resolver un problema de tamaño n . Determine cual es el mejor algoritmo para procesar $n = 2^{20}$ elementos si el algoritmo A tarda $10 \mu s$ para procesar 1024 items y B requiere de solo de $1 \mu s$ para la misma entrada.
4. Las librerías A y B son frecuentemente usadas para trabajar con grandes bases de datos de tamaño de 10^{12} registros. El procesamiento promedio con la librería A es $T_A(n) = 0,1n \log_2 n$ y usando la librería B es $T_B(n) = 5n \mu s$. Determine cual de las dos librerías tiene un mejor performance, indicando bajo que condiciones una es mejor que la otra.

2. Notación Asintótica

A. Diga, para cada afirmación, si es Verdadero o Falso (justifique si es Falso). Considere que f , g y h son funciones asintóticamente no negativas.

$n^{3,00001} \in O(n^3)$ $n^{2,9999} \in \Omega(n^3)$ $n^2 / \log n \in o(n^2)$ $n^5 \in \omega(n^2)$ $(\log n)^{\log n} \in o(n^{\log \log n})$ $\log n^2 + \log n \in \Theta(\log n^3)$ $O(f \cdot g) \in O(f) \cdot O(g)$ Si $g = O(f) \wedge h = O(f)$, entonces $g = O(h)$	$n^2 - 5n \in \Omega(n^2)$ $n^{10}/10000 \in o(n^{10})$ $n^2 \log n \in \omega(n^2)$ $(n+1)! \in \Omega(n!)$ $4^{\log n} \in \Theta(n^2)$ $O(f+g) \in O(f) + O(g)$ $5n + 7n^2 + 99n^3 \in O(n^2 \log n)$
---	--

B. Demuestre que:

1. Sean f_1 y f_2 funciones asintóticamente positivas, con $f_1(n) \in O(g_1(n))$ y $f_2(n) \in O(g_2(n))$. Pruebe entonces que $(f_1 + f_2)(n) \in O((g_1 + g_2)(n))$
2. $2^{n+1} \in O(2^n)$
3. $2^{2n} \in \Omega(2^n)$
4. $\log n \in O(n^k)$, $\forall k$ constante positiva
5. $\log n \in o(n)$
6. $(\log n)^2 \in o(n)$
7. Si $a > 1$ entonces $n \in o(a^n)$
8. $\sum_{i=1}^n \frac{1}{2^i} = 1$, con n grande

3. Recurrencias

Resuelva de manera exacta las siguientes recurrencias.

1. $T(n) = 3T(n-1) + 2$; $\forall n \geq 2$; $T(1) = 1$.
2. $T(n) = 4T(n/3) + n^2$; $\forall n \geq 3$ $T(1) = 1$; asuma que n es potencia de 3.
3. $T(n) = 3T(n/2) + n^2 - n$; $T(1) = 1$; asuma que n es potencia de 2.
4. $T(n) = 3T(n/2) + n^2 - 2n + 1$; $\forall n \geq 2$; $T(1) = 4$; asuma que n es potencia de 2.
5. $T(n) = T(n-2) + 3n + 4$; $\forall n \geq 3$; $T(1) = 1$, $T(2) = 6$.
6. $T(n) = \sum_{i=1}^{n-1} T(i) + 1$; $\forall n \geq 2$, $T(1) = 1$.

$$7. T(n) = \frac{1}{n} \sum_{i=0}^{n-1} T(i) + c \cdot n; T(0) = 0.$$

4. Teorema Maestro

El Teorema Maestro es aplicable en recurrencias de la forma:
 $T(n) = aT(n/b) + f(n)$, donde $a \geq 1$ y $b > 1$, y $f(n)$ es una función asintóticamente positiva (positiva cuando n es muy grande).

A. Para el caso particular y simplificado en que $f(n) = t \cdot n$, para algún $t > 0$. Se dan tres casos posibles:

- Si $a = b$, entonces $T(n) = \Theta(n \log n)$
- Si $a < b$, entonces $T(n) = \Theta(n)$
- Si $a > b$, entonces $T(n) = O(a^{\log_b n}) = O(n^{\log_b a})$

Dado el resumen anterior, aplique el Teorema Maestro para resolver las siguientes recurrencias:

1. $T(n) = 16T(n/4) + n$
2. $T(n) = 3T(n/2) + n$
3. $T(n) = 4T(n/2) + c \cdot n$
4. $T(n) = 3T(n/3) + n/2$

B. Para el caso general en la forma de $f(n)$, reconocemos tres casos posibles:

- Si $f(n) = O(n^{\log_b a - \epsilon})$ para algún $\epsilon > 0 \Rightarrow T(n) = \Theta(n^{\log_b a})$
- Si $f(n) = \Theta(n^{\log_b a} \log^k n)$, $k \geq 0 \Rightarrow T(n) = \Theta(n^{\log_b a} \cdot \log^{k+1} n)$
- Si $f(n) = \Omega(n^{\log_b a + \epsilon}) \wedge a \cdot f(n/b) \leq c \cdot f(n)$, para algún $\epsilon > 0$ y $c < 1$
 $\Rightarrow T(n) = \Theta(f(n))$

Para este caso general, aplique el Teorema Maestro, cuando sea posible, para resolver las siguientes recurrencias:

1. $T(n) = 3T(n/2) + n^2$
2. $T(n) = 4T(n/2) + n^2$

3. $T(n) = T(n/2) + 2^n$
4. $T(n) = 2T(n/2) + n \log n$
5. $T(n) = 2T(n/2) + n/\log n$
6. $T(n) = 6T(n/3) + n^2 \log n$

5. Problemas Algorítmicos

1. De una relación de recurrencia para el problema de buscar un elemento en un arreglo ordenado a través de la clásica búsqueda binaria, resuelva y obtenga su complejidad.
2. Considere la siguiente variante al algoritmo `mergeSort(A, n)`; donde A es la secuencia de elementos de largo n que se desea ordenar (asuma n potencia de 3):


```

mergeSort(A, n){
  if  $n = 1$  then return  $A$ 
  else{
    divida  $A$  en 3 partes iguales:  $X, Y, Z$ 
    mergeSort( $X, n/3$ )
    mergeSort( $Y, n/3$ )
    mergeSort( $Z, n/3$ )
    return merge( $X, Y, Z$ )
  }
}
```

 donde la función `merge( $A_1, A_2, A_3$ )` hace y devuelve, en orden lineal, la mezcla de las tres sublistas ordenadas. Determine y resuelva la relación de recurrencia para el peor caso en el número de comparaciones de esta versión del algoritmo.
3. Considere los siguientes tres algoritmos que resuelven un mismo problema P de tamaño n :
 - 1- Algoritmo X , que resuelve el problema dividiéndolo en cinco subproblemas de la mitad del tamaño, luego recursivamente resuelve cada uno de ellos para finalmente combinar las soluciones en tiempo lineal.
 - 2- Algoritmo Y , que resuelve P dividiéndolo en nueve subproblemas de tamaño $n/3$, resolviendo recursivamente cada uno de ellos y combinando las soluciones en tiempo $O(n^2)$.
 - 3- Algoritmo Z , que resuelve P recursivamente con dos subproblemas de tamaño $n-1$ y luego combina las soluciones en tiempo constante.
 ¿Cuál es el tiempo de ejecución de cada uno? ¿Cuál escogería Ud.?
4. Indique cual es la complejidad (en el número de comparaciones) del siguiente segmento de código en C++:

```

for( $i = 1; i < n; i++$ ){
  for( $j = n; j > 0; j = j/2$ ){
    for( $k = 0; k < n; k = k + 2$ ){
      ... //un número constante de operaciones
    }
  }
}

```

5. Para el siguiente segmento de código en C++, indique cuantas veces se imprime el saludo (puede asumir que n es una potencia de 2):

```

for( $i = 1; i < n; i = i * 2$ ){
  for( $j = 0; j < i; j++$ ){
    cout << "saludo"
  }
}

```

6. Indique cuantas operaciones primitivas se ejecutan en el siguiente segmento de código en C++ (en términos de n , a y b); luego de la complejidad asintótica en términos de n , considerando que a y b son valores constantes:

```

for( $k = 1; k < n; k = k * 2$ ){
  for( $i = 0; i < k; i = i + 1$ ){
    for( $j = 1; j < n; j = j * 2$ ){
      ... //a operaciones primitivas
    }
    for( $j = 0; j < n; j = j + 2$ ){
      ... //b operaciones primitivas
    }
  }
}

```

Luego calcule el número total y exacto entre comparaciones (de los ciclos for) y las operaciones primitivas.

7. Considere el siguiente algoritmo que recibe como parámetro un valor $m = 2^n$.

```

algoritmo( $m$ ){
   $p = 2$ 
  for  $i \leftarrow 1$  to  $\log m$  then
     $p = p * p$ 
  return  $p$ 
}

```

- i- ¿Qué valor devuelve el algoritmo para el m de entrada?
 ii- ¿Cuál es su tiempo de computo en función de n ?

8. Proponga un algoritmo que permita remover elementos repetidos de un arreglo desordenado de números $A[1..n]$ en tiempo $O(n \log n)$, quedando $A[1..n - k]$ después de eliminar los k repetidos. No importa el orden que quede en el arreglo final, lo importante es que pueda remover los repetidos.
9. Para el mismo problema de eliminar elementos repetidos en $A[1..n]$, suponga ahora que $\max - \min = c \cdot n$; para alguna constante pequeña $c > 0$ y donde $\max$ y $\min$ son el máximo/mínimo valor en A respectivamente. Si dispone de un espacio adicional de $c \cdot n$ palabras en la RAM, ¿Podría resolver este problema en un tiempo de $O(n)$?
10. Suponga que se tiene un arreglo A de tamaño infinito, sus primeras n celdas tienen enteros positivos ordenados estrictamente. El resto de las celdas un -1. Un detalle es que no se sabe cuál es el valor de n .
¿Es posible determinar el valor de n en tiempo $O(\log n)$?
11. ¿Cuántas líneas imprime la siguiente función?

```

funcion( $n$ ){
  if  $n > 1$  then {
    imprimir-linea("una más")
    funcion( $n/2$ )
    funcion( $n/2$ )
  }
}
```

6. Algunas Respuestas

Sección 1

1. Formula explícita $T(n) = \frac{1}{3}10^{-9}n \log n$
Para 10^6 items, $T(n) = 2\text{ms}$.
2. El algoritmo B es asintóticamente mejor (más chico) que A ; y a partir de $n_0 \geq 10^{25}$ A siempre toma más tiempo que B .
Si $n \leq 10^9$, el mejor algoritmo es A .
4. En términos de complejidad asintótica, $T_B(n) = 5n$ es inferior a $T_A(n) = 0,1n \log_2 n$; lo que en la práctica se cumple cuando $0,1 \log_2 n \geq 5$, esto es $n \geq 2^{50}$. Por tanto, cuando se tienen hasta $n \leq 2^{50}$ items, la mejor alternativa es la librería A .

Sección 2.B

1. Por definición $f_1(n) \leq c_1 g_1(n)$ y $f_2(n) \leq c_2 g_2(n)$, con c_1, c_2 positivos.
Sea $c = \max\{c_1, c_2\}$ entonces $f_1(n) + f_2(n) \leq c(g_1(n) + g_2(n))$.
3. Por demostrar que $\exists c > 0, n_0 \in \mathbb{N}$ tal que: $2^{2n} \geq c \cdot 2^n \forall n \geq n_0$.
 $2^{2n} = 2^n 2^n \geq c \cdot 2^n \forall c \geq 1$ y $n \geq n_0 = 1$.
5. Queremos demostrar que $\log(n) \in o(n) \Leftrightarrow \exists n_0 > 0$ tal que $\forall c > 0$
 $0 \leq \log n \leq c \cdot n, \forall n \geq n_0$
Luego, es fácil ver que

$$\lim_{x \rightarrow \infty} \frac{\log x}{x} = 0$$

, y por tanto siempre existe algún natural $n_0 > 0$ tq: $c \geq \frac{\log n}{n}$ a partir de todo $n \geq n_0$ sea cual sea la constante positiva c .

Sección 3

1. $T(n) = 3T(n-1) + 2; \forall n \geq 2; T(1) = 1$
 $T(n) = 3T(n-1) + 2$ reemplazando el término $T(n-1)$, queda...
 $T(n) = 3(3T(n-2) + 2) + 2 = 3^2 T(n-2) + 2(3^0 + 3^1)$
 $T(n) = 3^2(3T(n-3) + 2) + 2(3^0 + 3^1)$
 $= 3^3 T(n-3) + 2(3^0 + 3^1 + 3^2)$
...
 $T(n) = 3^k T(n-k) + 2 \sum_{i=0}^{k-1} 3^i$; Luego, si $k = n-1 \Rightarrow T(n-k) = T(1)$; quedando:
 $T(n) = 3^{n-1} T(1) + 2 \sum_{i=0}^{n-2} 3^i$

$$T(n) = 3^{n-1} + \frac{2(3^{n-1}-1)}{2}$$

$$T(n) = 2 \cdot 3^{n-1} - 1$$

Ahora falta demostrar el paso inductivo, que $T(n+1) = 2 \cdot 3^n - 1$

Dem.

$$T(n+1) = 3T(n) + 2 = 3(2 \cdot 3^{n-1} - 1) + 2 = 2 \cdot 3^n - 1.$$

Por tanto Q.E.D.

2. $T(n) = 4T(n/3) + n^2$; $\forall n \geq 3$ $T(1) = 1$; asuma que n es potencia de 3.

$$\text{Respuesta: } T(n) = 4^{\log_3 n} + \frac{1-(4/9)^{\log_3 n}}{5/9} n^2$$

Sección 4.B

1. Si $T(n) = 3T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2)$ (caso 3).
2. Si $T(n) = 4T(n/2) + n^2 \Rightarrow T(n) = \Theta(n^2 \log n)$ (caso 2).
4. Si $T(n) = 2T(n/2) + n \log n \Rightarrow T(n) = n \log^2 n$ (caso 2).